Relatório de Execução de Testes: API e Plataforma Web

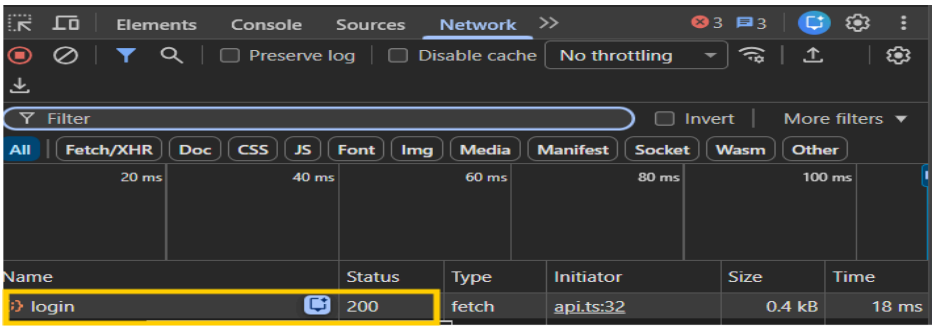
Ambiente de Teste & Credenciais Utilizadas

- **Administrador (Admin):** Login: `admin` | Senha: `admin123`
- **Usuário Comum:** Login: `user1` | Senha: `pass123`

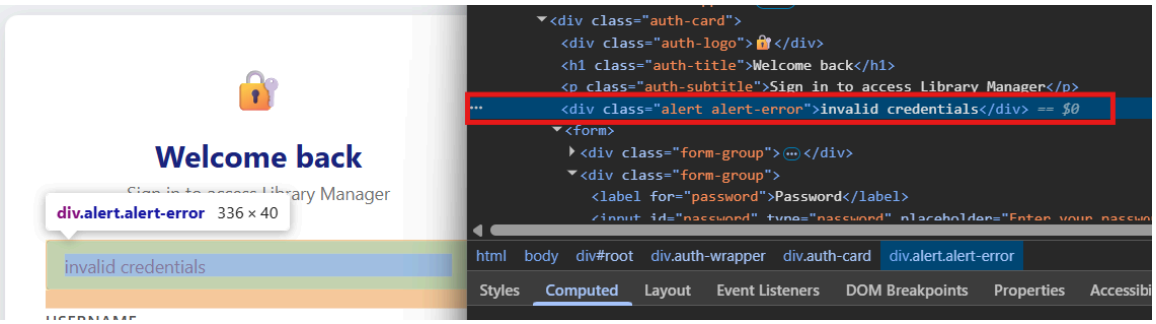
1. Testes de Autenticação e Gestão de Acessos

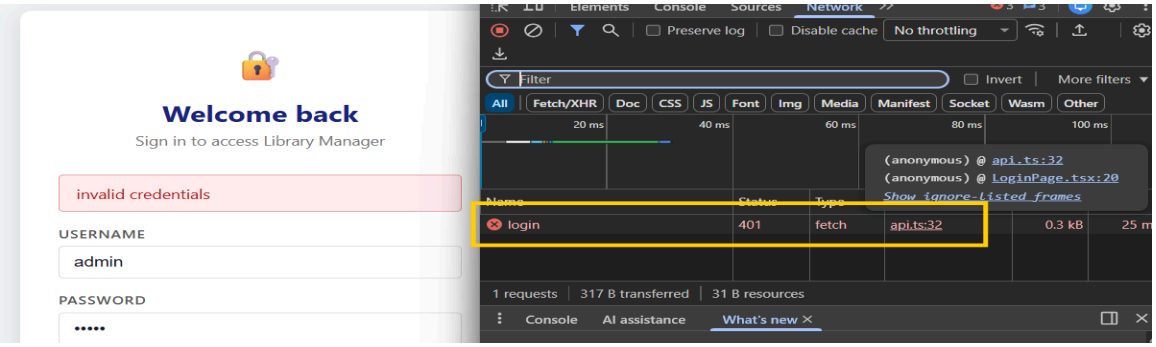
Fluxo de Login: Perfil Administrador (Admin)

- **Caso de Sucesso:** Comportamento da Interface de Usuário (UI) operando em conformidade. A API valida as credenciais e retorna o status `200 OK` (JWT token issued successfully).



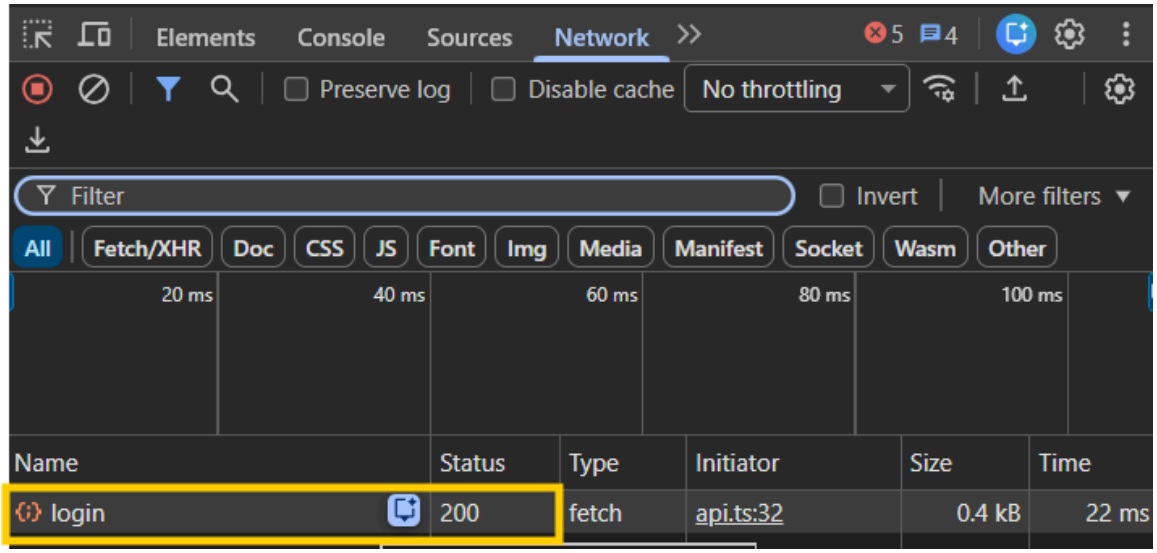
- **Caso de Erro (Credenciais Inválidas):** Front-end exibe o alerta de erro esperado de forma clara. A API barra o acesso com segurança, retornando `401 Unauthorized` (Teste realizado com senha incorreta



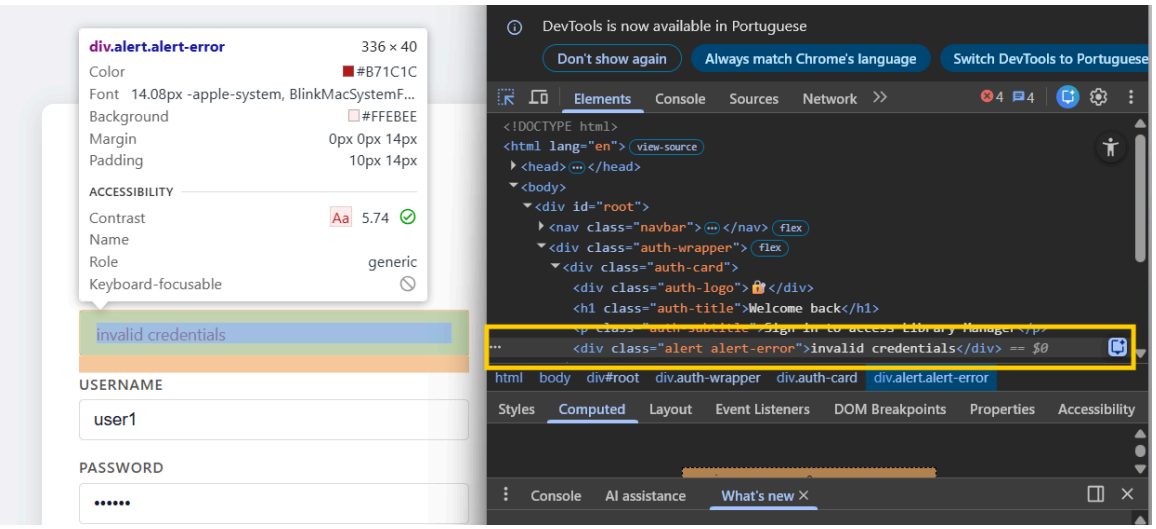


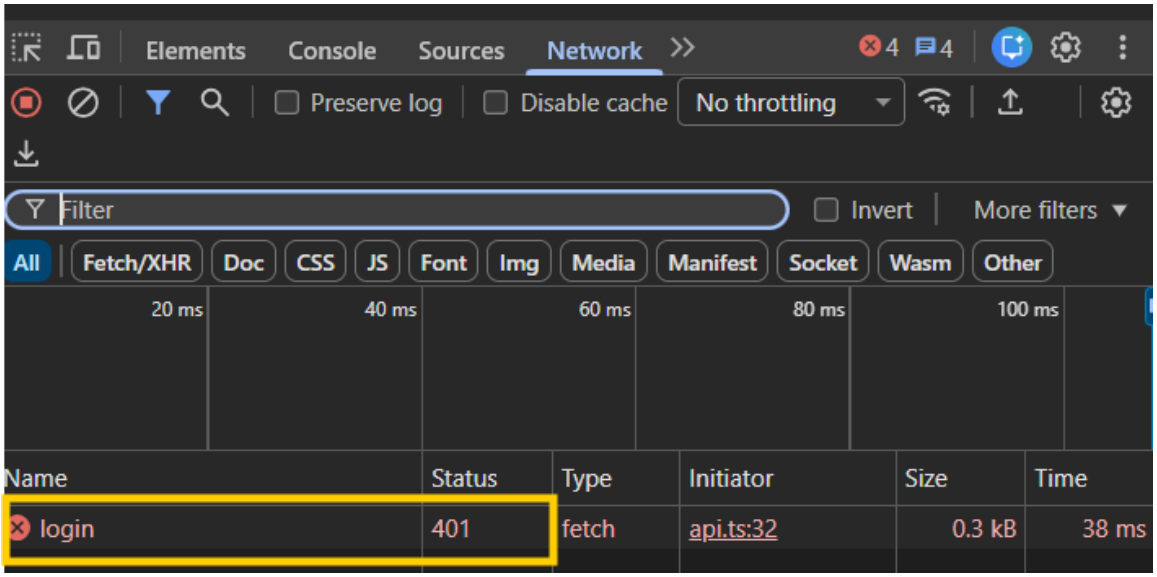
Fluxo de Login: Perfil Usuário Comum (User)

- **Caso de Sucesso:** UI renderiza a entrada no sistema sem anomalias. API atende a requisição validando o acesso e retornando `200 OK` (JWT token issued successfully).



- **Caso de Erro (Credenciais Inválidas):** A UI informa corretamente que a credencial está incorreta. A API protege a aplicação e retorna o status `401 Unauthorized`.





Fluxo de Criação de Novos Acessos (Web e Swagger)

- **Caso de Sucesso:** Ao registrar um novo usuário, a API responde corretamente com `201 Created` (User created successfully).

Name	Status	Type	Initiator	Size	Time
register	204	preflight	Preflight	0.0 kB	37 ms
register	201	fetch	api.ts:43	0.3 kB	4 ms

Curl

```
curl -X 'POST' \
  'http://localhost:8080/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "role": "admin",
  "username": "vitoriaadmin",
  "password": "vitoria123"
}'
```

Request URL

```
http://localhost:8080/auth/register
```

Server response

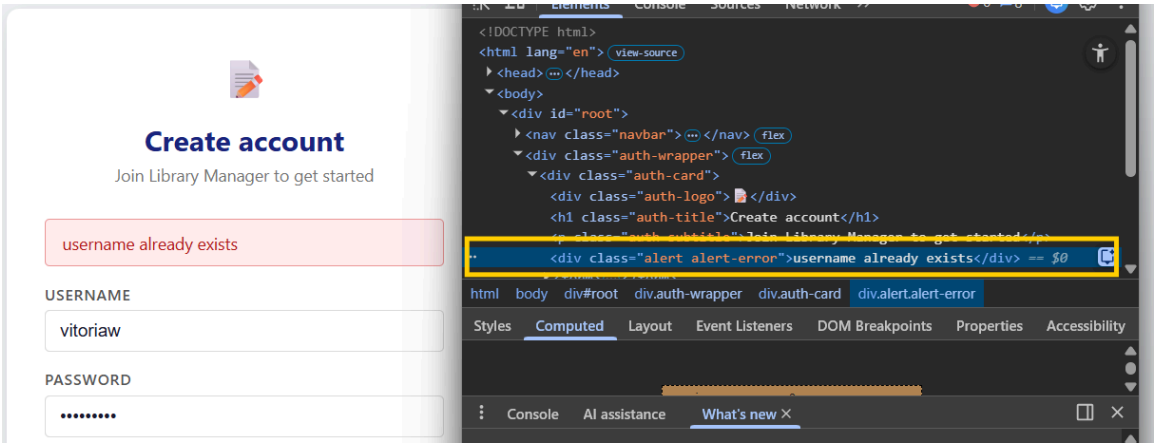
Code	Details
201	Response body

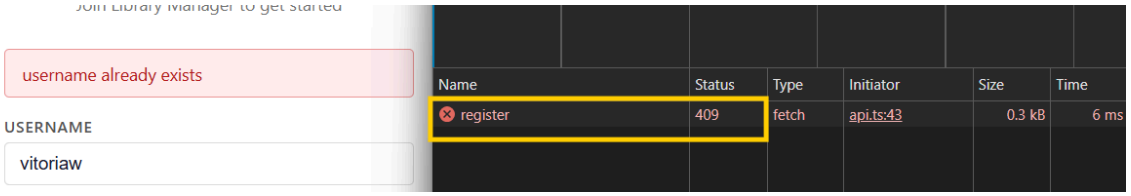
```
{
  "id": 6,
  "role": "user",
  "username": "vitoriaadmin"
}
```

Response headers

```
content-length: 48
content-type: application/json; charset=utf-8
date: Thu,02 Apr 2026 05:38:57 GMT
```

- **Caso de Erro (Login Repetido/Duplicado):** A UI exibe a mensagem de conflito esperada. A API blinda o banco de dados retornando `409 Conflict` (Username already exists). *Nota: Foi realizado teste de validação cruzada mantendo o login e alterando a senha, com o bloqueio ocorrendo com sucesso.*





Curl

```
curl -X 'POST' \
  'http://localhost:8080/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "vitoria2",
  "password": "vitoria123"
}'
```

Request URL

```
http://localhost:8080/auth/register
```

Server response

Code	Details
409	Error: Conflict

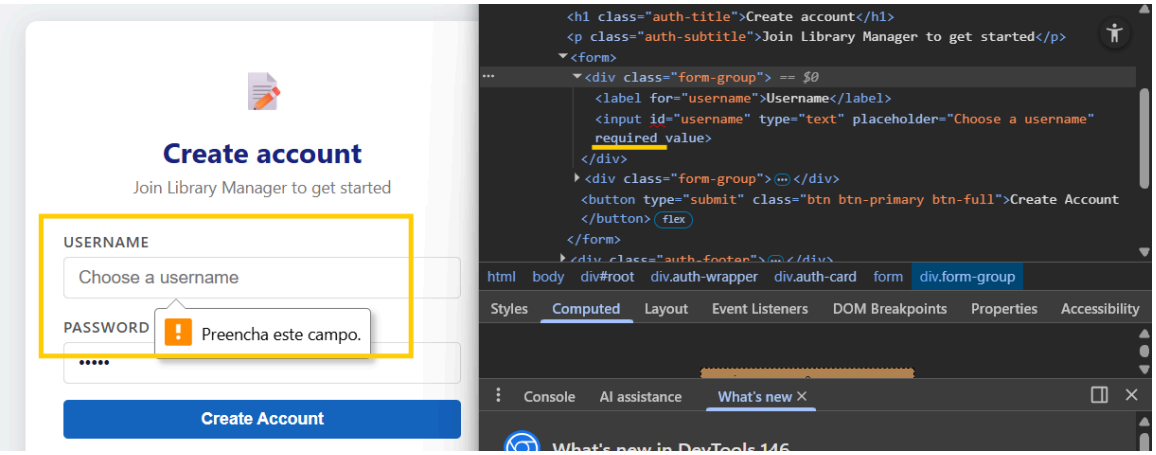
Response body

```
{
  "error": "username already exists"
}
```

Response headers

```
content-length: 35
content-type: application/json; charset=utf-8
date: Thu,02 Apr 2026 05:14:55 GMT
```

- **Caso de Erro (Credencial Ausente):** Validação de obrigatoriedade do Front-end (atributo `required`) bloqueia o envio e exibe "Preencha este campo". Em testes diretos via Swagger (burlando o Front-end), a API cobre a falha e retorna `400 Bad Request` (Missing fields).



Curl

```
curl -X 'POST' \
  'http://localhost:8080/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "",
  "password": "vitoria123"
}'
```

Request URL

```
http://localhost:8080/auth/register
```

Server response

Code	Details
400	Error: Bad Request

Response body

```
{
  "error": "username and password are required"
}
```

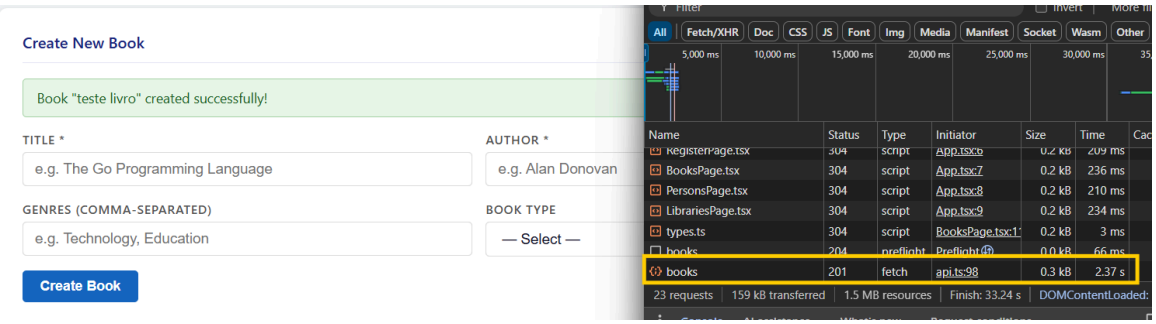
Response headers

```
content-length: 46
content-type: application/json; charset=utf-8
date: Thu,02 Apr 2026 05:41:39 GMT
```

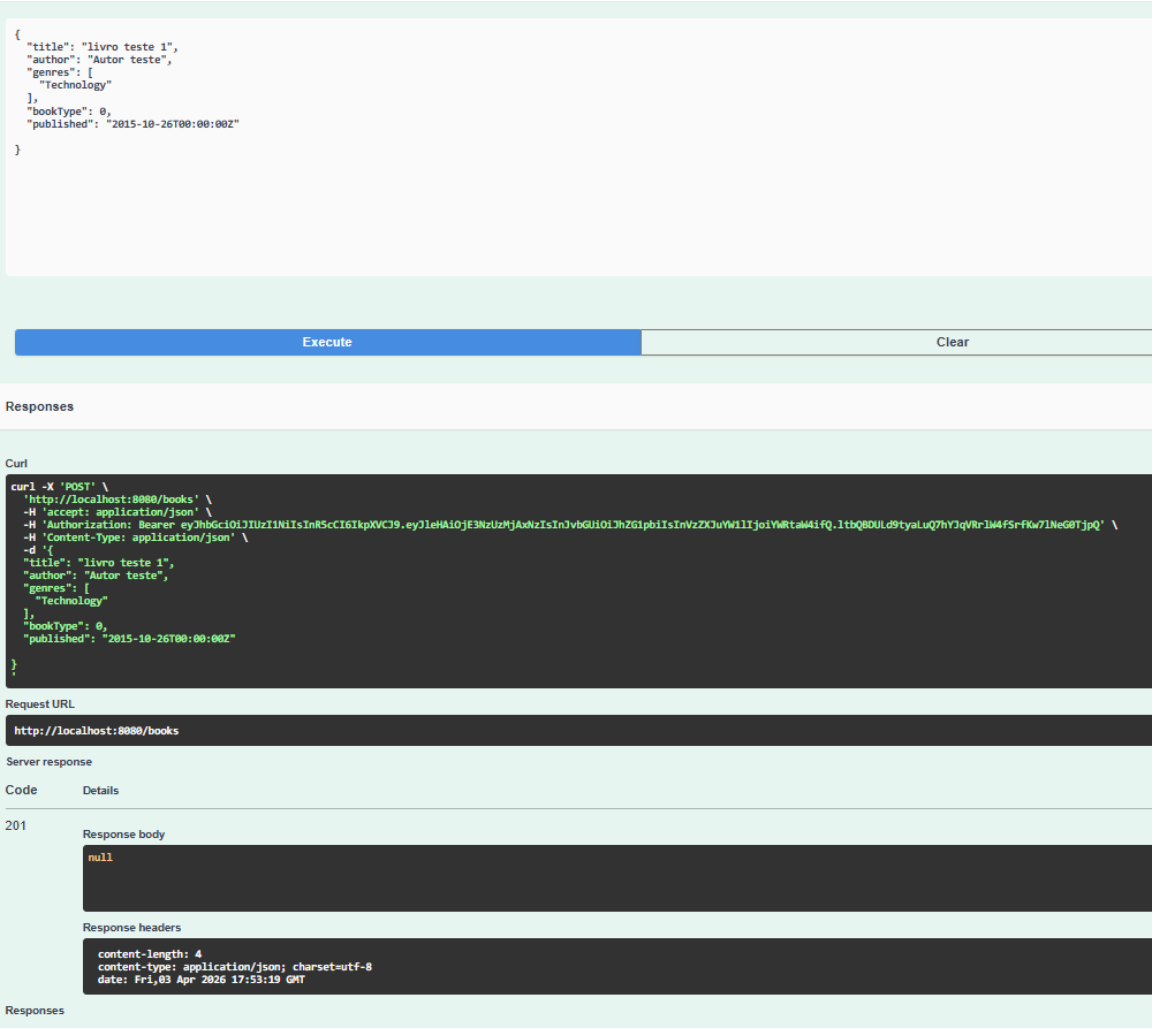
2. Fluxos Operacionais: PERFIL ADMINISTRADOR (Admin)

Gestão de Livros (Books)

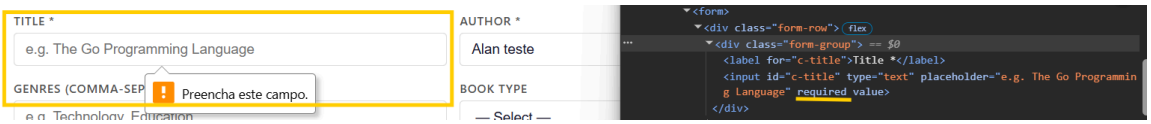
- **Criar Novo Livro - Sucesso (Requisitou Hotfix QA):** O fluxo original apresentava falhas de código impeditivas. Após a aplicação de um hotfix corretivo no Back-end por esta analista, o sistema passou a criar livros com sucesso, retornando `201 Created`. *[https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-001.pdf]*.

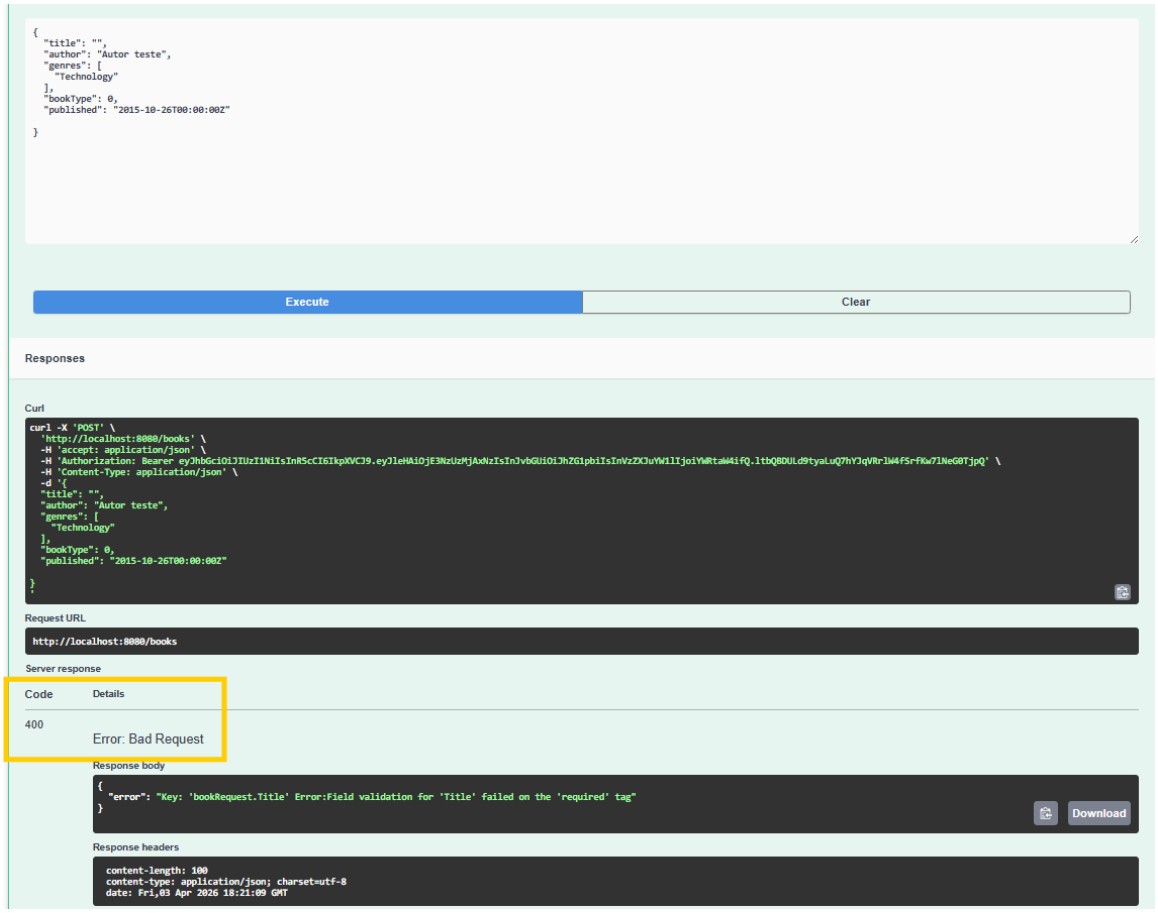


*A criação só foi possível ser feita após o login ativo pelo swagger no try it out e inserindo o token no Authorize.

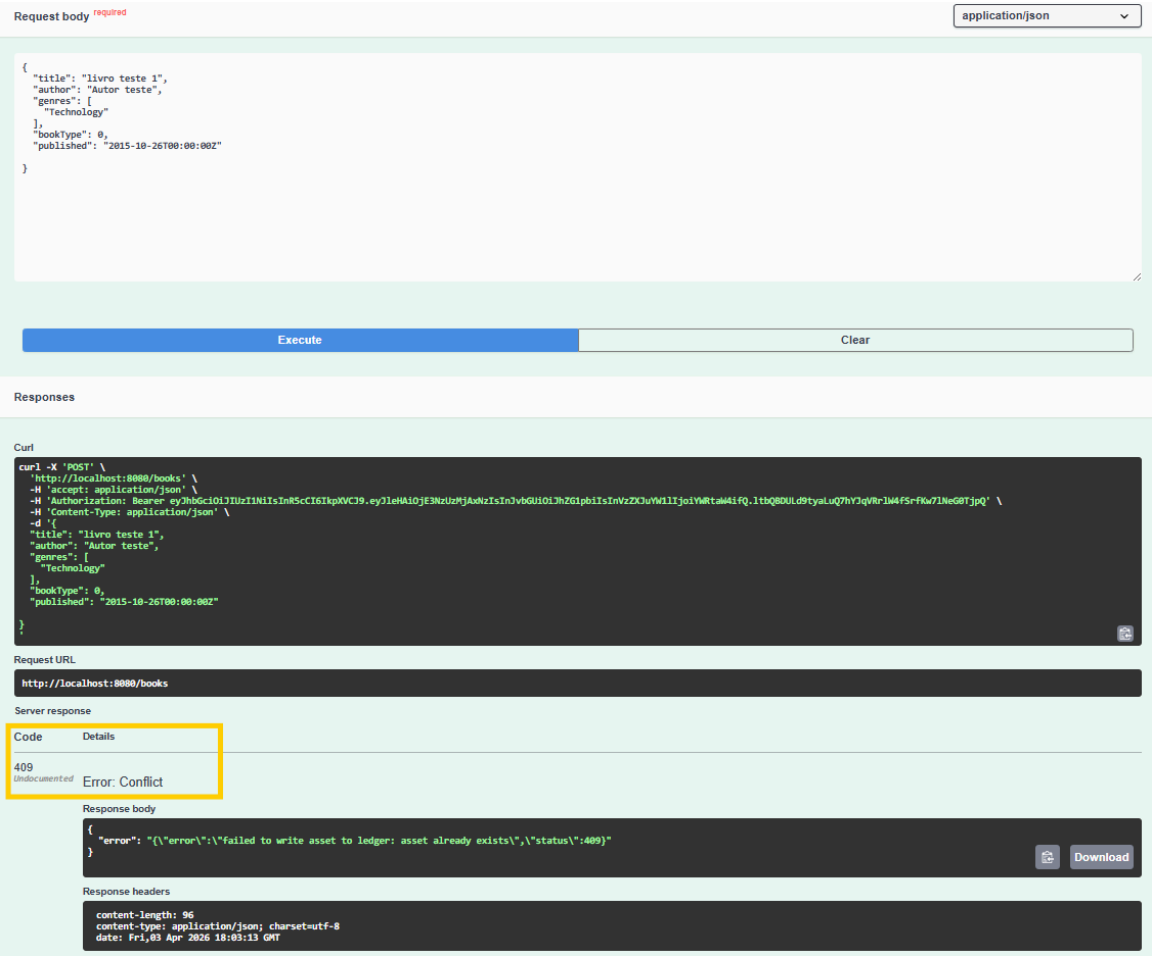


- **Criar Novo Livro - Falhas Tratadas:** Submissões com campos em branco são corretamente barradas pelo `required` do Front-end e retornam `400 Bad Request` via Swagger.

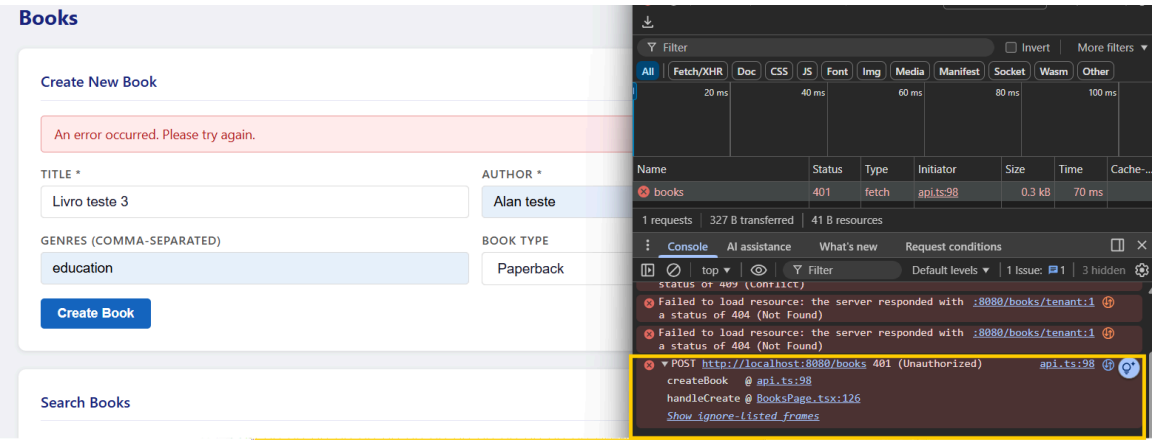




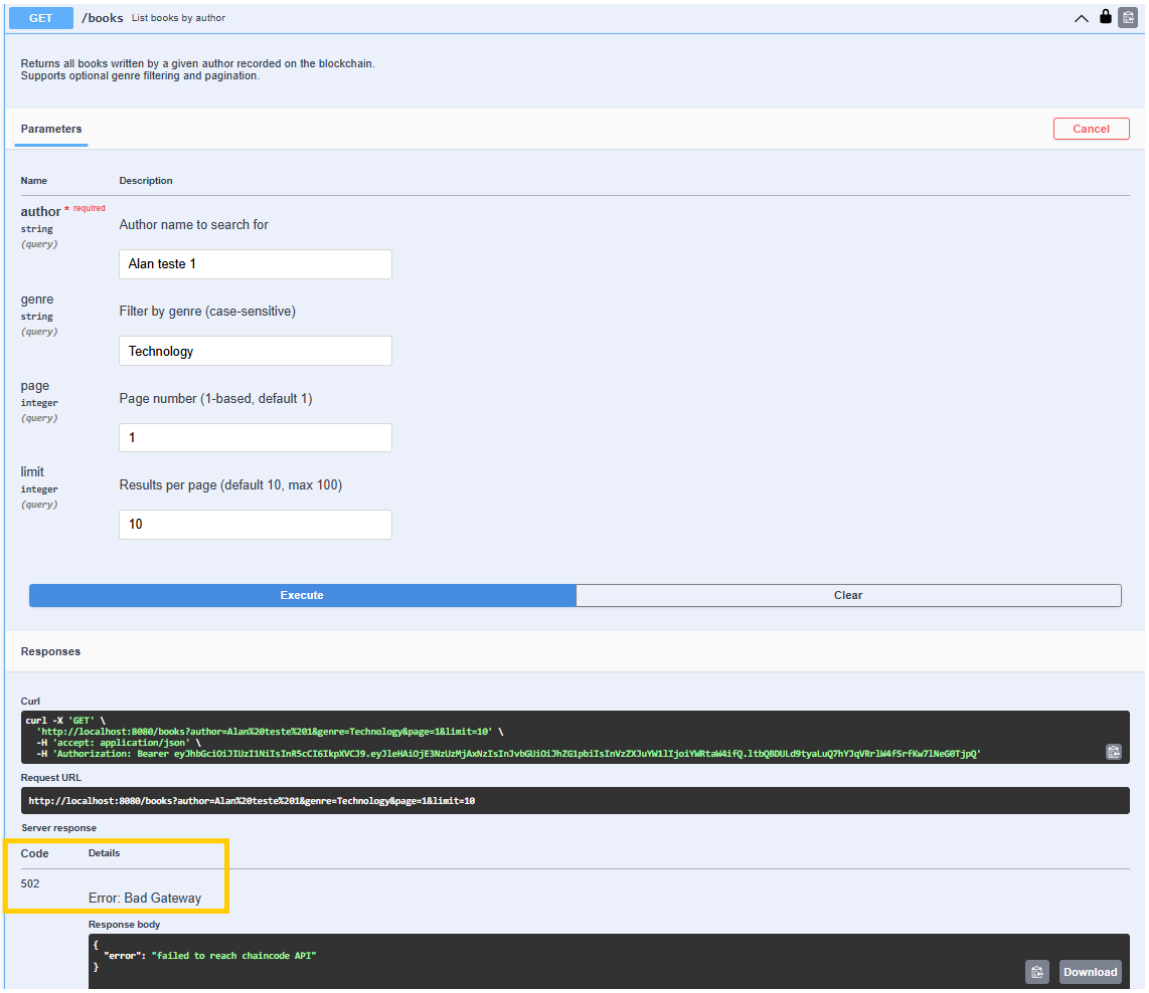
- **Criar Novo Livro - Validação de Duplicidade (Bug Impeditivo na Web / Divergência de Contrato):** Pela plataforma Web, não foi possível testar e validar o comportamento da interface ao tentar cadastrar um livro já existente devido a um bug impeditivo no fluxo. No entanto, ao contornar a UI e realizar o teste isolado via Swagger, o sistema bloqueou a criação do item repetido com sucesso. Constatou-se, porém, uma quebra de contrato na resposta: a documentação especifica o erro `400 Bad Request`, mas a API retorna `409 Conflict`.



- **Criar Novo Livro - Falha de Autorização (401 Unauthorized):** Mesmo com perfil Admin ativo e dados válidos, a criação é bloqueada. O Front-end falha ao enviar o Token JWT para a API e a UI exibe apenas uma mensagem de erro genérica, sem tratamento adequado para perda de sessão. *[https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-002.pdf]*

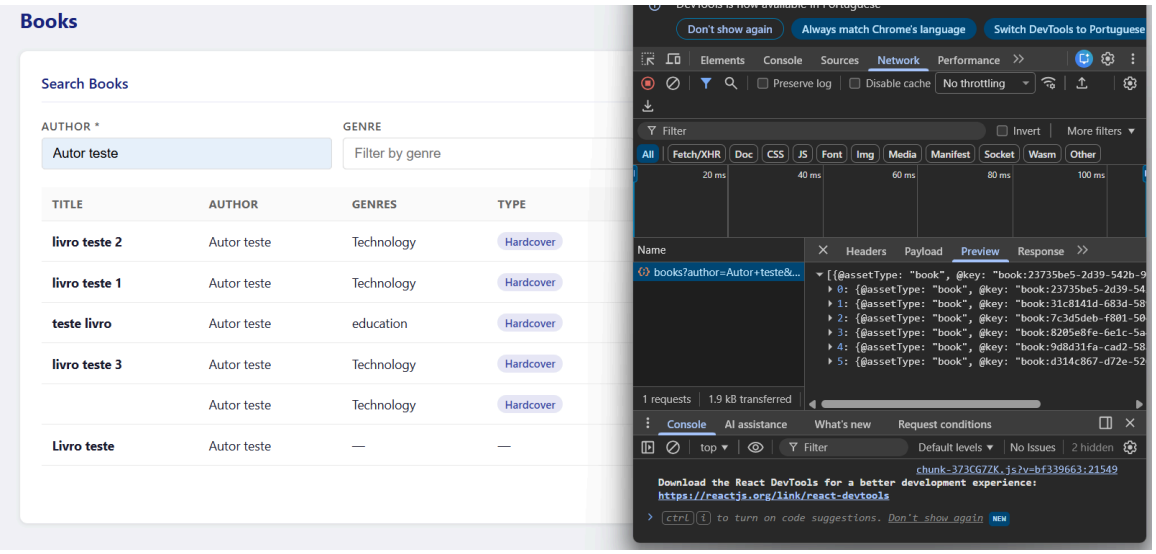


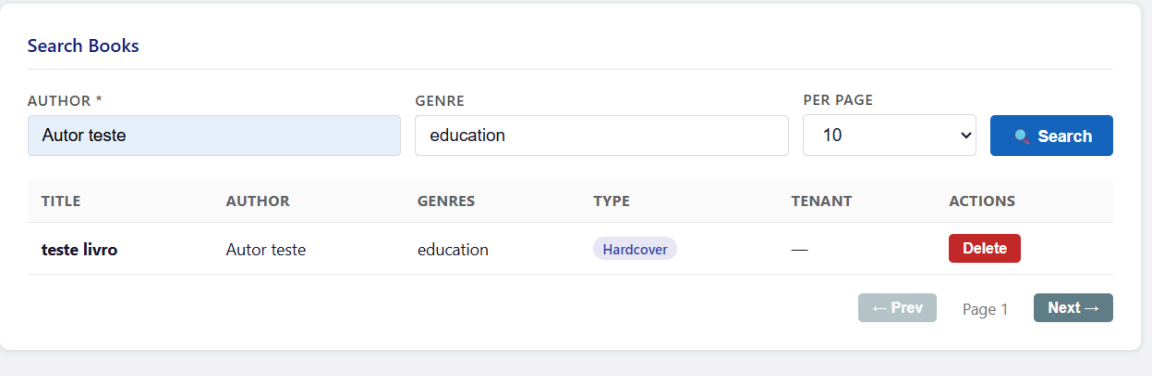
- **Simulação de Indisponibilidade (Falha de Conexão com AWS/Chaincode):** Para validar a resiliência e o tratamento de erros do sistema, foi injetada uma falha intencional de infraestrutura, alterando a URL de integração no arquivo `.env` para um servidor inexistente (`http://servidor-falso-da-vitoria.com`). O sistema lidou com a indisponibilidade de forma segura e controlada, com a API retornando o erro `502 Bad Gateway` (Could not reach the chaincode API), correspondendo exatamente ao comportamento esperado para este cenário.



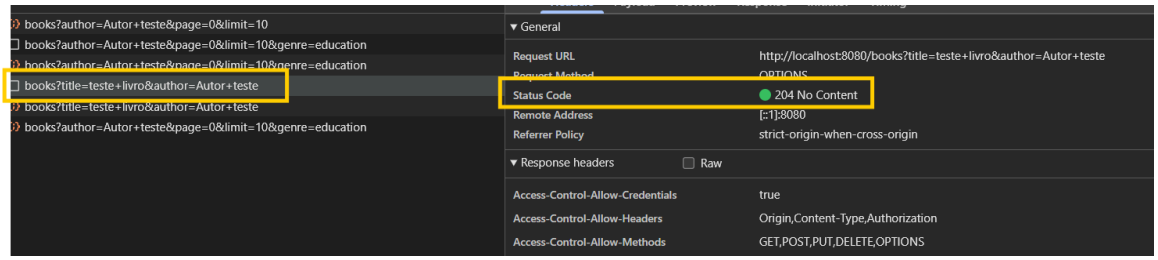
- **Buscar e Listar Livros (Search) - Sucesso (Requisitou Hotfix QA):** A princípio, a funcionalidade não estava operando corretamente devido a uma falha no código ao aplicar dois parâmetros de busca simultâneos. Para evitar que este erro atuasse como um impeditivo (blocker) na validação dos demais fluxos (tanto pela Web quanto pelo Swagger), foi realizada uma alteração prévia no sistema. Após a correção, a API passou a retornar `200 OK` e o Front-end exibiu com sucesso todos os livros

cadastrados anteriormente.
[https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-004.pdf]





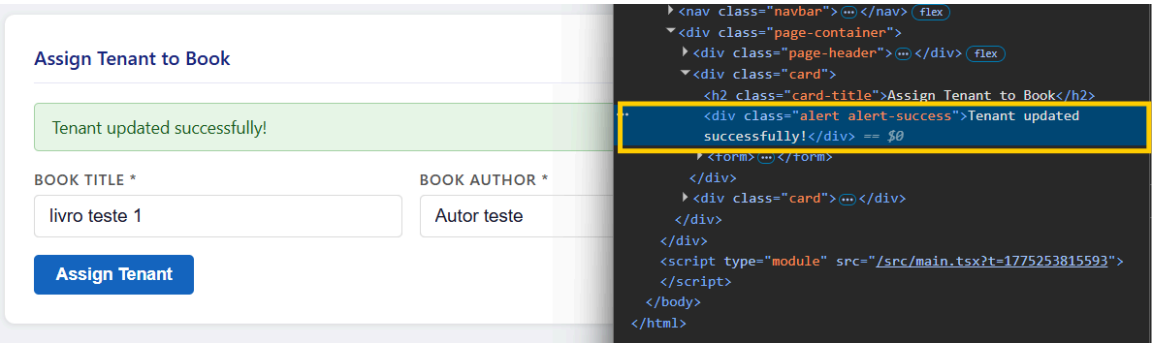
- **Excluir Livro (Delete) - Sucesso:** Viabilizado após o desbloqueio do fluxo de listagem (Search), o teste da função de exclusão pôde ser executado e validado. Na plataforma Web, o ativo deixa de ser exibido na interface imediatamente após a ação, e a API processa a requisição retornando o status `204 No Content`, funcionando exatamente de acordo com o comportamento esperado.

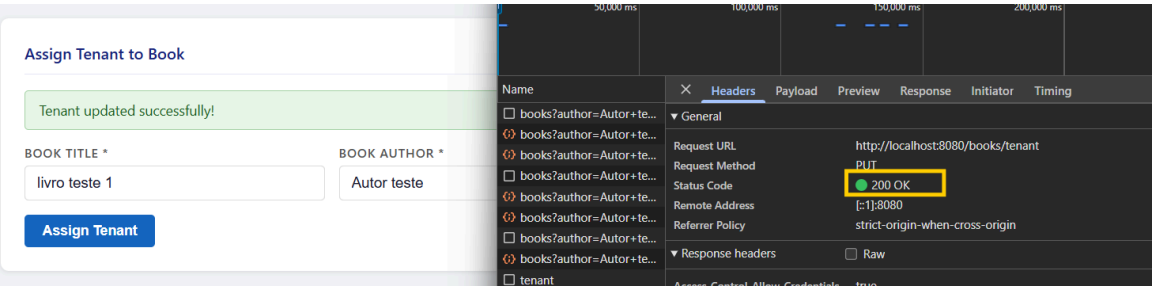


Atribuição de Locatário (Assign Tenant)

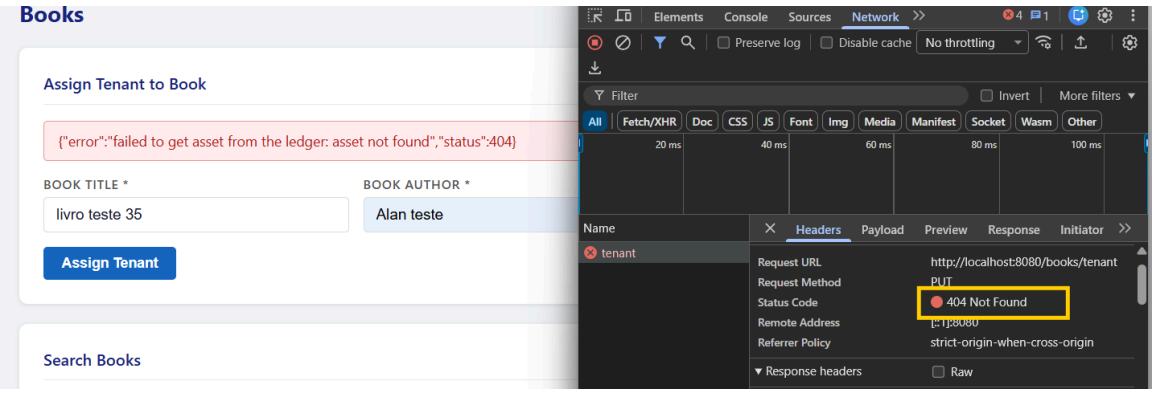
- **Atribuir Locatário - Falso Positivo (Erro Crítico de Integração):** Ao preencher os dados solicitados válidos (Título do Livro, Autor e CPF do Locatário), a interface exibe

a mensagem de sucesso e a API retorna `200 OK`. No entanto, ao realizar a validação de persistência no banco (via pesquisa), constata-se que o vínculo não foi salvo de fato. Trata-se de uma falha silenciosa crítica na camada de integração. *[https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-007.pdf]*

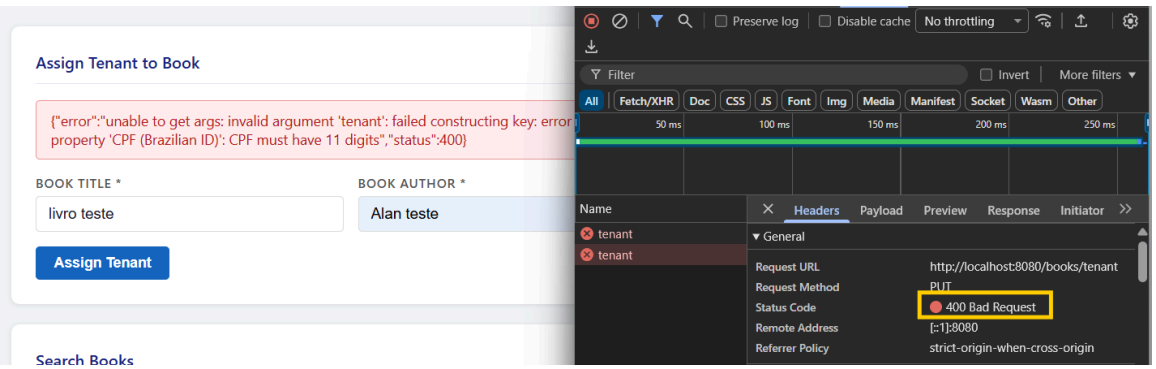




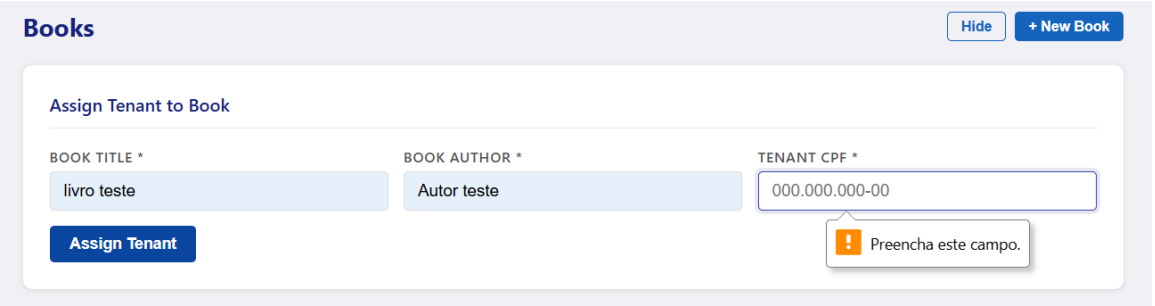
- **Atribuir Locatário - Ativos Inexistentes:** O teste foi realizado inserindo o nome de um livro ou autor não cadastrados previamente na plataforma. O Front-end tratou a exceção exibindo a mensagem de erro esperada e a API bloqueou a ação corretamente, retornando o status `404 Not Found`.



- **Atribuir Locatário - Validação de Formato (CPF Inválido):** Ao inserir uma quantidade de números superior ao padrão exigido para o CPF brasileiro, o sistema atua em conformidade. O Front-end renderiza a mensagem de alerta e a API rejeita o payload retornando o erro `400 Bad Request`.

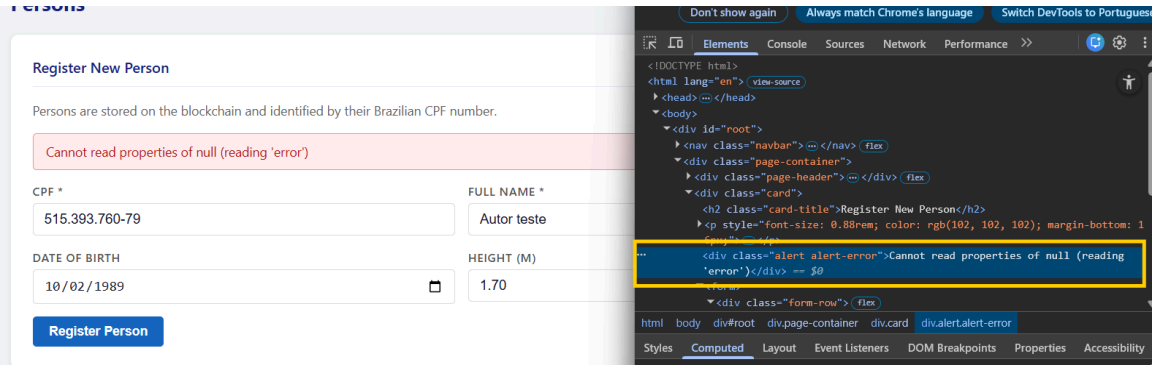


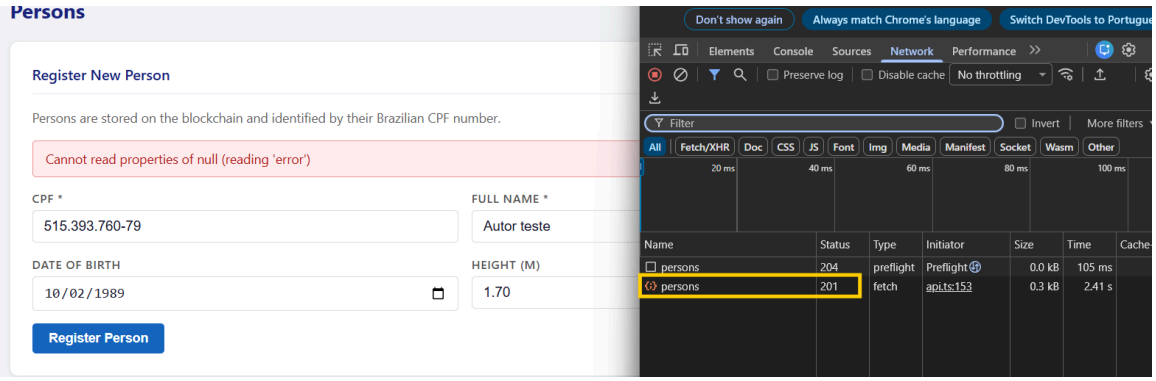
- **Atribuir Locatário - Ausência de Dados (Campos Vazios):** A validação visual de obrigatoriedade (atributo `required`) está funcionando corretamente no Front-end, bloqueando o envio do formulário e impedindo que requisições incompletas sejam disparadas para a API.



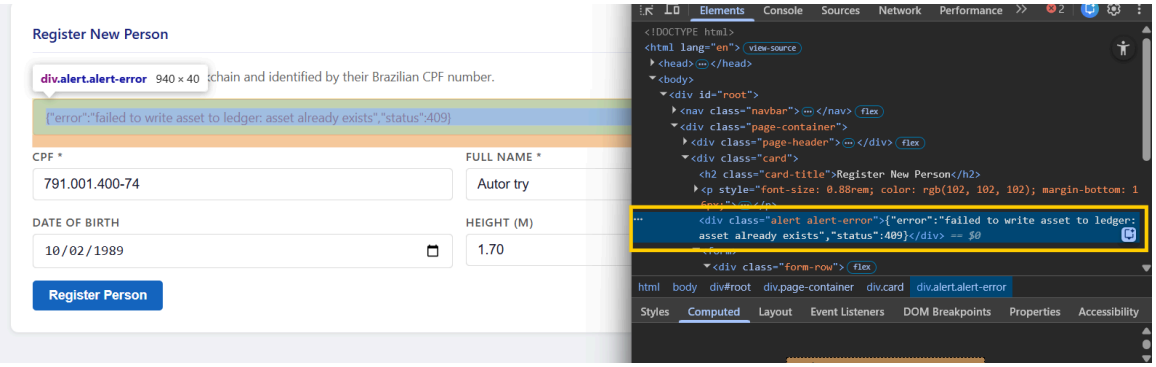
Gestão de Pessoas (Persons)

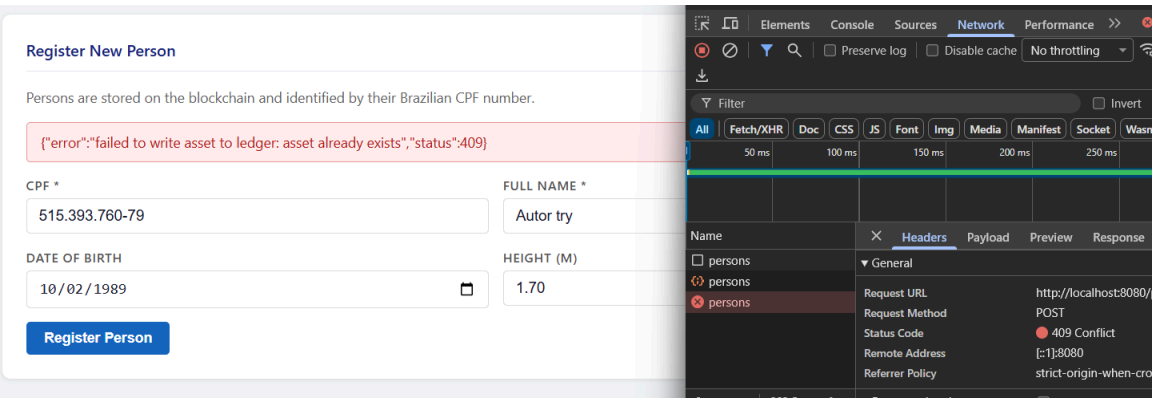
- **Registrar Pessoa - Falso Negativo na UI (Bug Visual):** Ao realizar o cadastro com dados válidos, a API processa a requisição com sucesso e retorna `201 Created`, conforme o esperado. Contudo, há uma falha de tratamento no Front-end que exibe um alerta de "Erro" indevido para o usuário ao invés da mensagem de confirmação de sucesso. *[https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-006.pdf]*



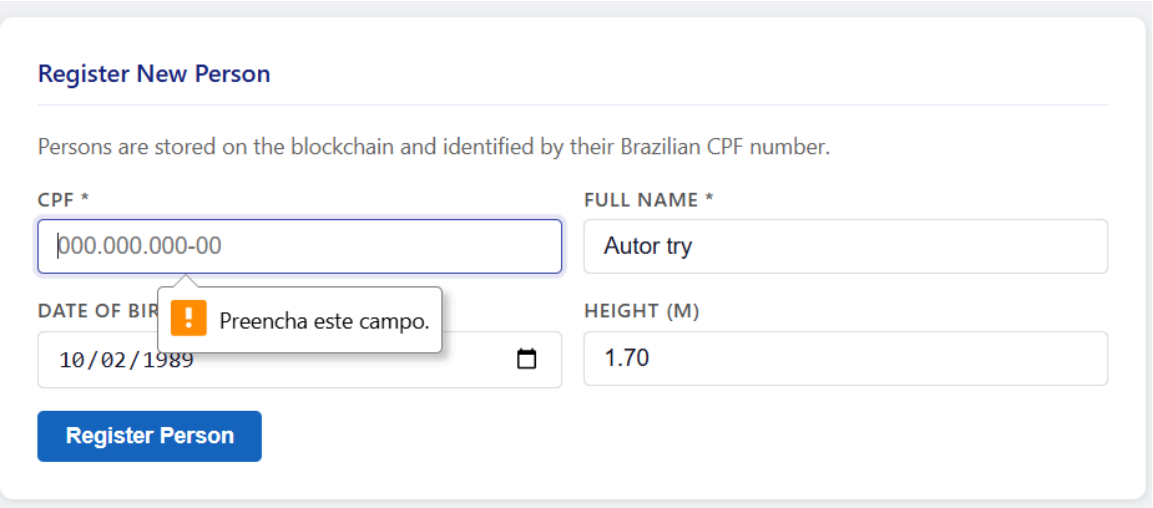


- **Registrar Pessoa - Validação de Duplicidade (CPF Cadastrado):** Na tentativa de registrar um usuário utilizando um CPF que já consta na base de dados, o sistema reage corretamente. A mensagem de erro é exibida de forma adequada na interface e a API atua na proteção dos dados retornando o status `409 Conflict`.



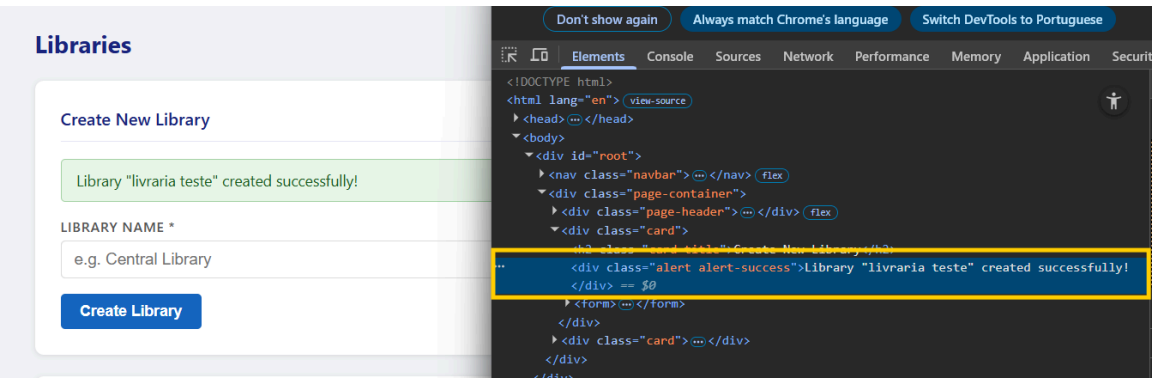


- **Registrar Pessoa - Ausência de Dados (Campos Vazios):** A validação dos campos obrigatórios (CPF e Nome Completo) está operando de forma correta no Front-end, impedindo o envio do formulário incompleto.



Gestão de Bibliotecas (Libraries)

- **Criar Biblioteca - Erro de Integração e Sucesso Pós-Correção (Requisitou Hotfix QA):** O fluxo original apresentava um defeito crítico de integração (JSON binding) entre o Front-end e a API, resultando em falha na submissão e retorno do erro `400 Bad Request`. Para avançar com os testes deste módulo, foi necessária uma correção técnica no código-fonte do Back-end. Após o hotfix, a operação passou a funcionar perfeitamente: a mensagem de sucesso é exibida na tela, a API retorna `201 Created` e a nova biblioteca é devidamente persistida no banco de dados (comportamento validado também via Swagger). *[https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-008.pdf]*



Name

libraries

libraries

Headers | Payload | Preview | Response | Initiator | Timing

▼ General

Request URL	http://localhost:8080/libraries
Request Method	POST
Status Code	201 Created
Remote Address	[::1]:8080
Referrer Policy	strict-origin-when-cross-origin

```
{
  "name": "livraria teste 3"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/libraries' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3NzUzMjAxNzIsInJvbGUiOiJhZG1pbiIsInVzZXJuYW1lIjoiYWRtaW4ifQ.1tbQBDULd9tyaLuQ7hYJqVRr1W4fSrfKw71N
  -H 'Content-Type: application/json' \
  -d '{
  "name": "livraria teste 3"
}'
```

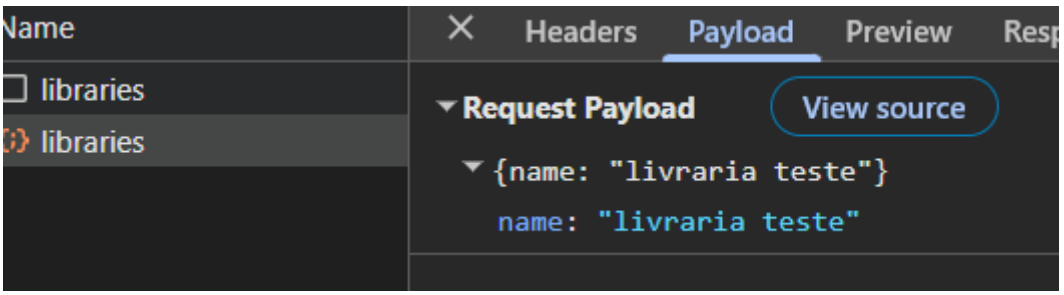
Request URL

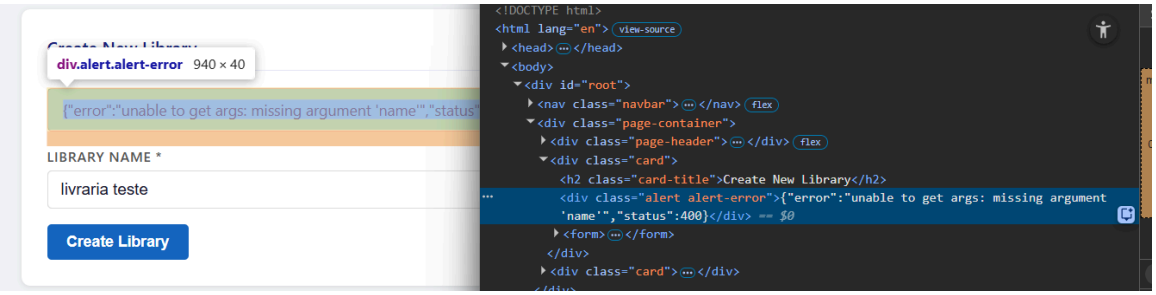
```
http://localhost:8080/libraries
```

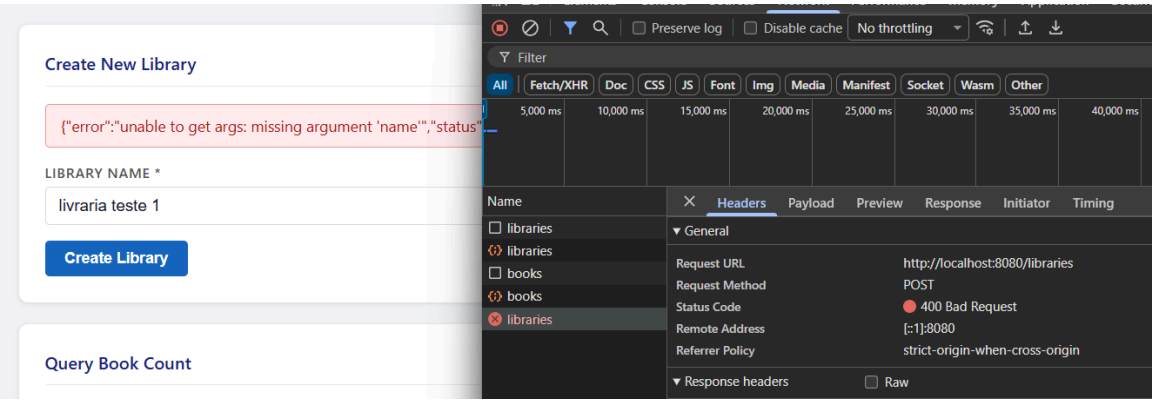
Server response

Code	Details
201	Response body

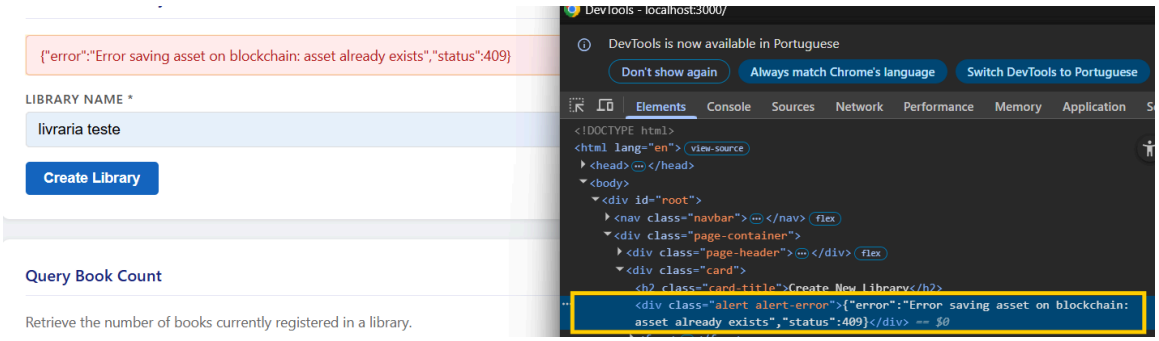
```
{
  "@assetType": "library",
  "@key": "library:319af86f-5e05-5d8a-b953-7fdcf7fd4556",
  "@lastTouchBy": "ProcessoSeletivoQAMSP",
  "@lastTx": "createNewLibrary",
  "@lastUpdated": "2026-04-04T15:33:31Z",
  "name": "livraria teste 3"
}
```

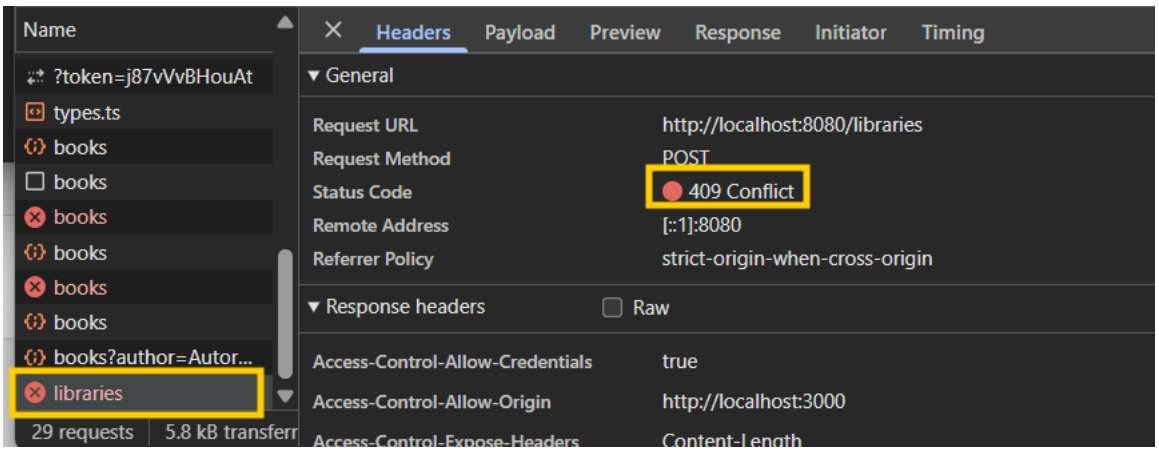


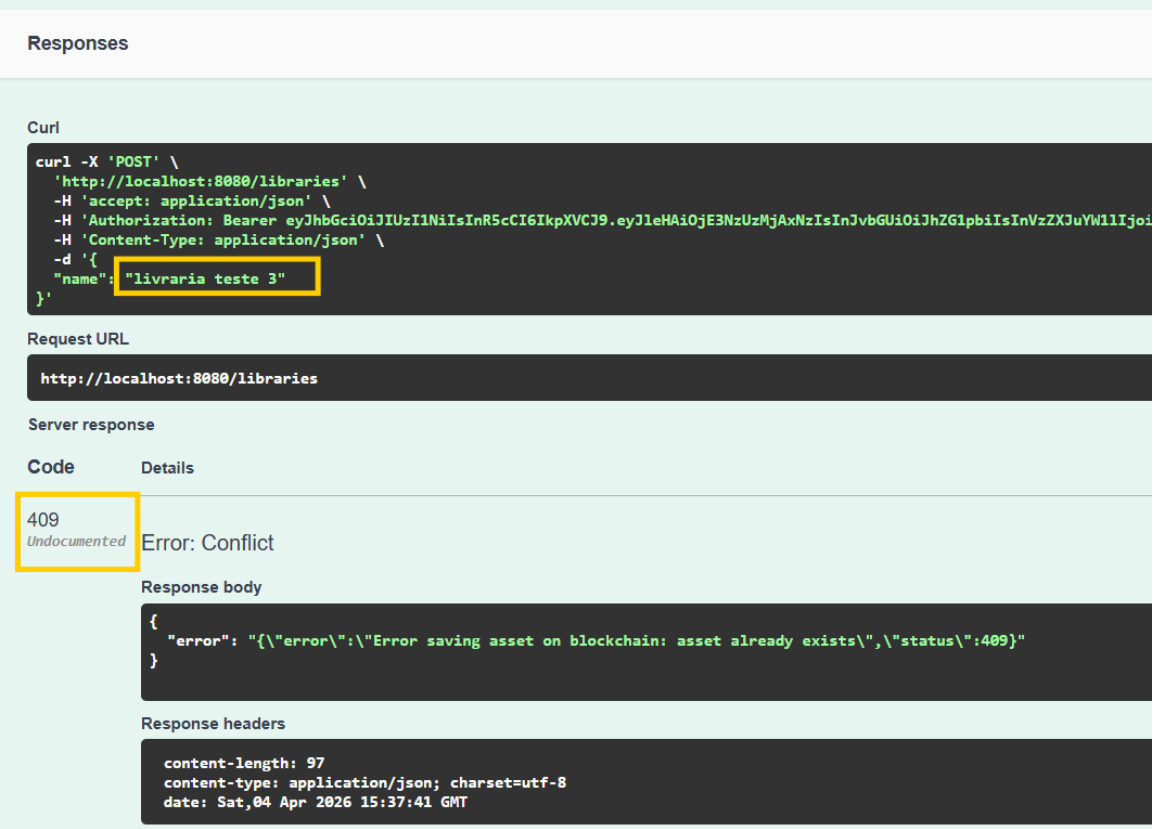




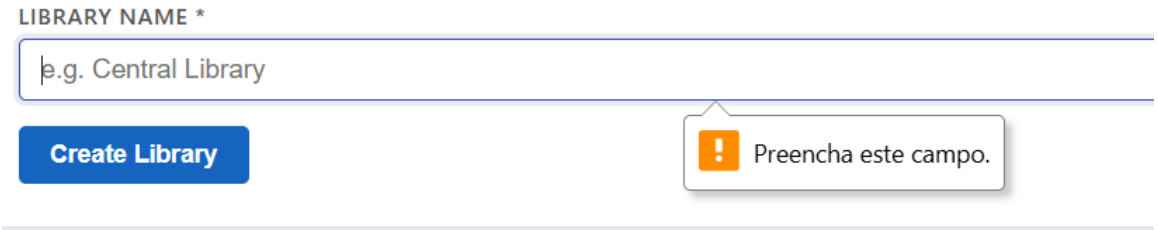
- **Criar Biblioteca - Validação de Duplicidade:** Tentativas de cadastrar uma biblioteca com um nome já existente no sistema (testado também diretamente via Swagger) acionam corretamente a trava de segurança da API, que retorna o status `409 Conflict`.





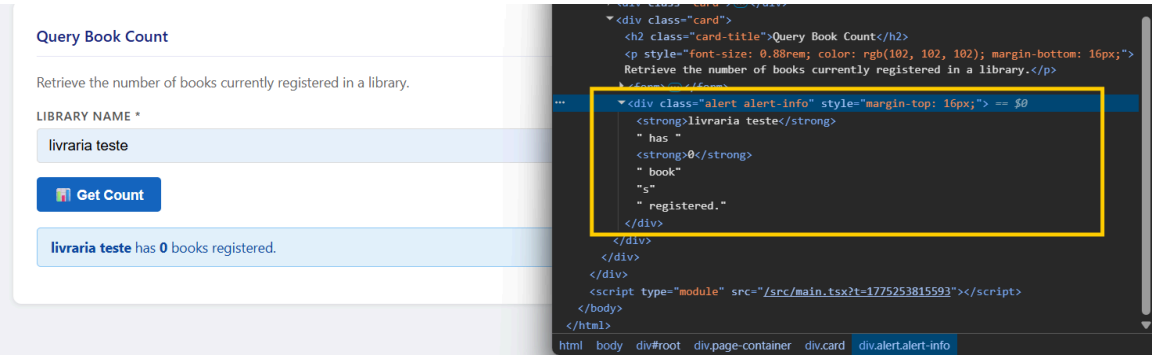


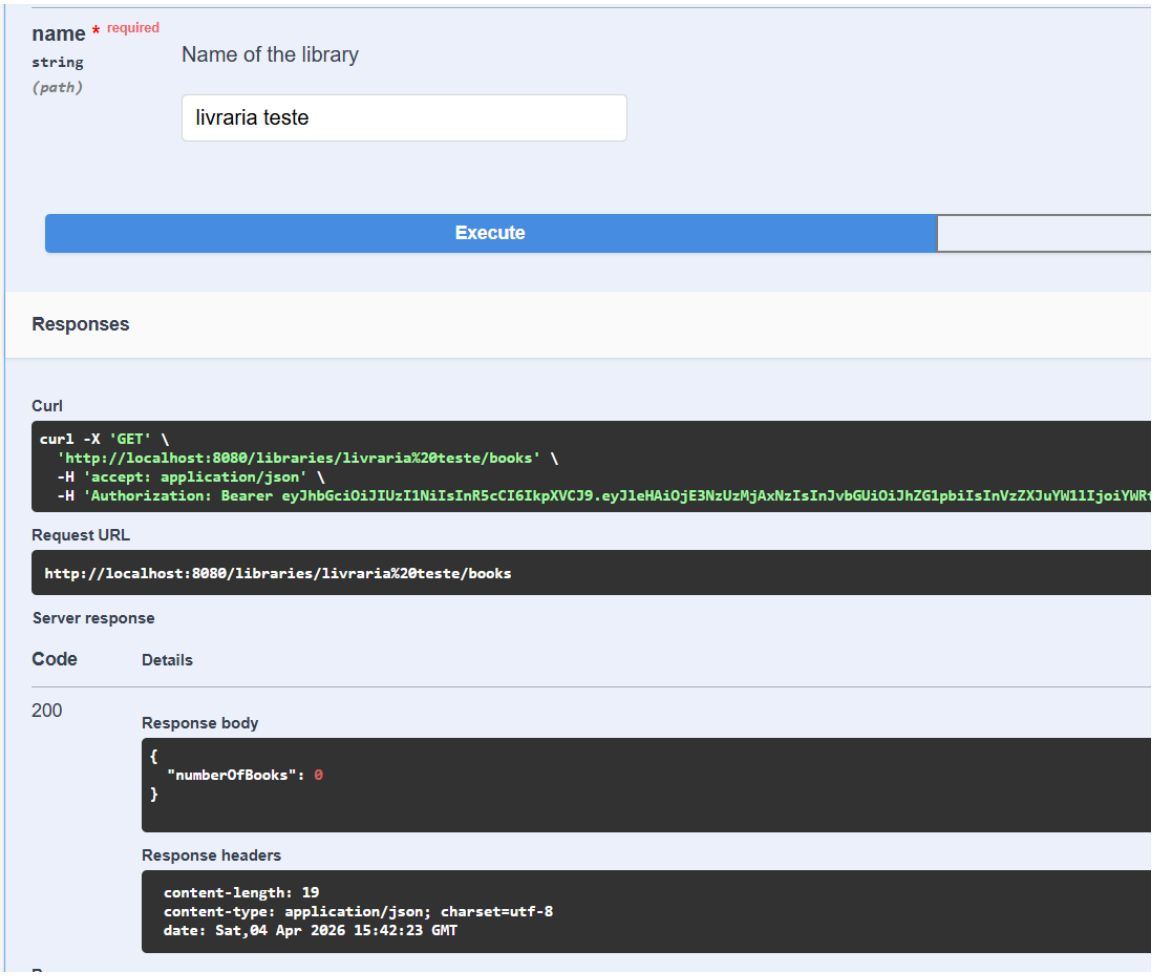
- **Criar Biblioteca - Ausência de Dados (Campo Vazio):** A validação visual de campo obrigatório (`required`) atua com precisão, bloqueando o clique e impedindo que requisições nulas ou incompletas sobrecarreguem o servidor.



- **Consultar Quantidade de Livros (Query Book Count) - Sucesso:** Funcionalidade validada com êxito após o desbloqueio prévio do fluxo de criação. Ao inserir o nome

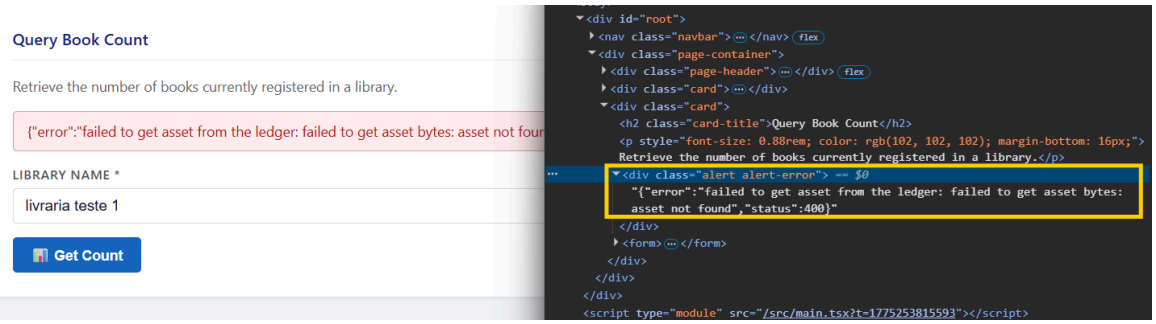
de uma biblioteca válida, a interface processa e exibe a mensagem de contagem correta, com a API retornando `200 OK` (comportamento espelhado nos testes via Swagger).

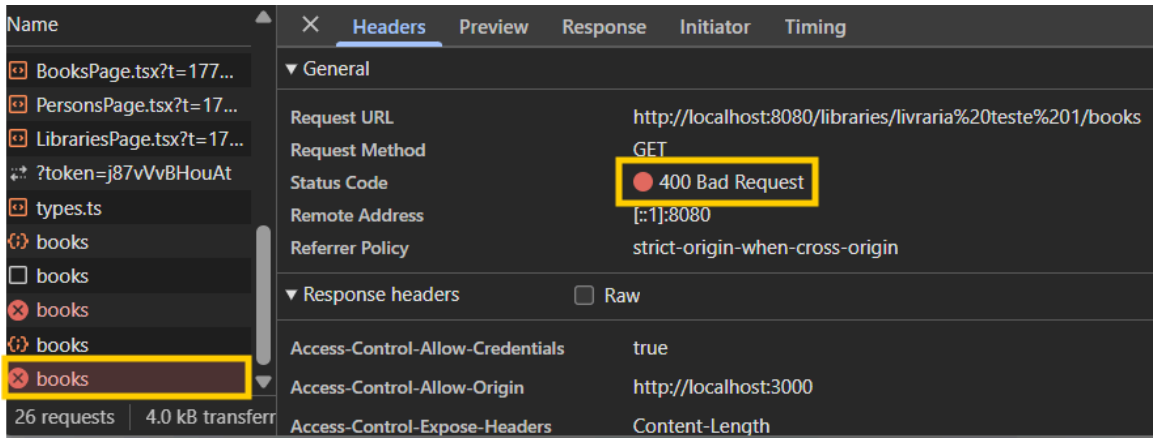


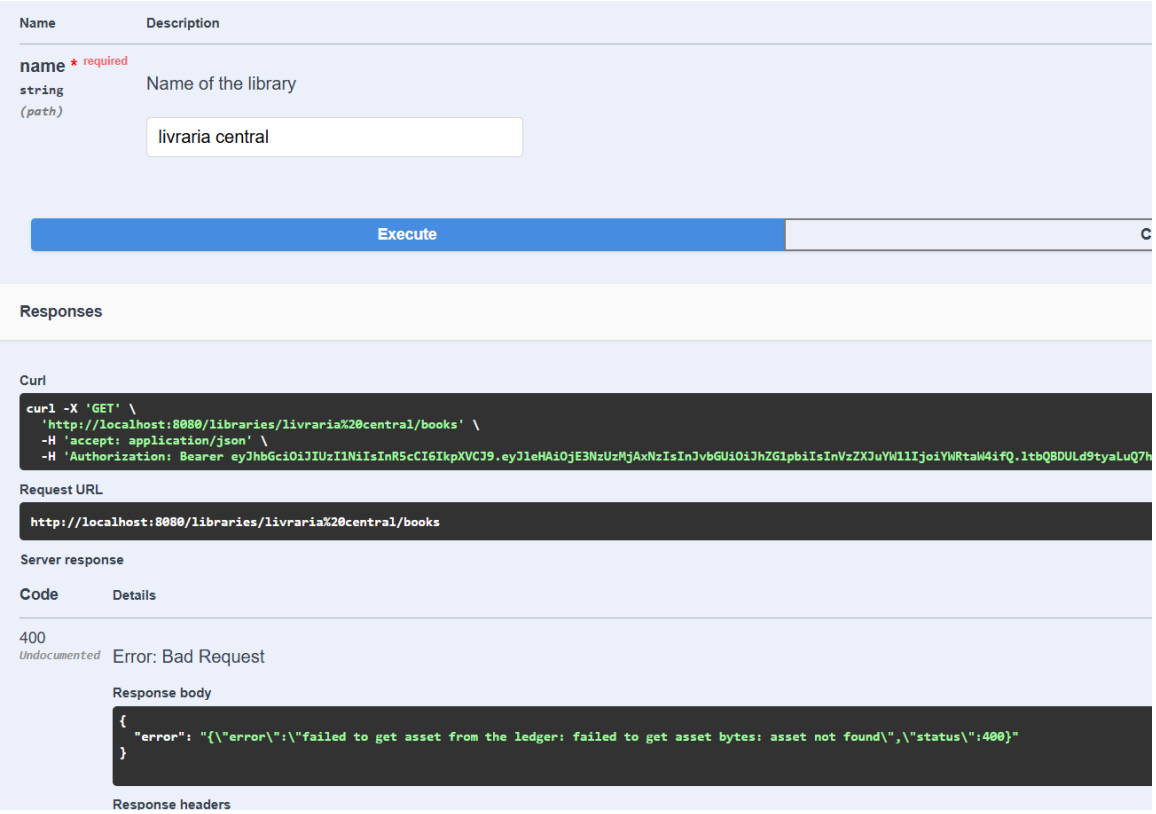


- **Consultar Quantidade de Livros - Ativo Inexistente:** Ao pesquisar por uma biblioteca que não consta no banco de dados, a interface exibe a mensagem de erro

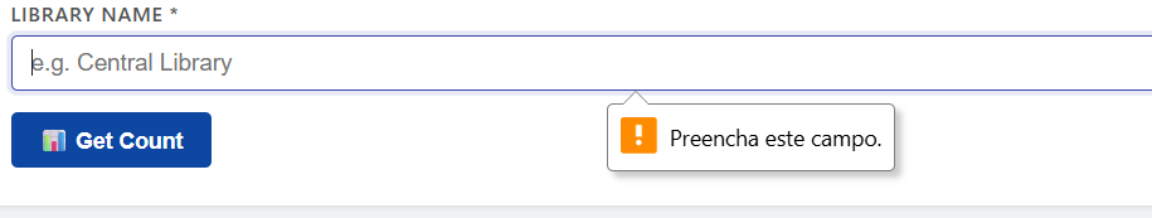
esperada. A API responde com `400 Bad Request`, obedecendo estritamente ao contrato estabelecido no Swagger. *(Nota de QA: Reitera-se a recomendação de melhoria arquitetural para adequação ao padrão REST `404 Not Found` neste cenário).*

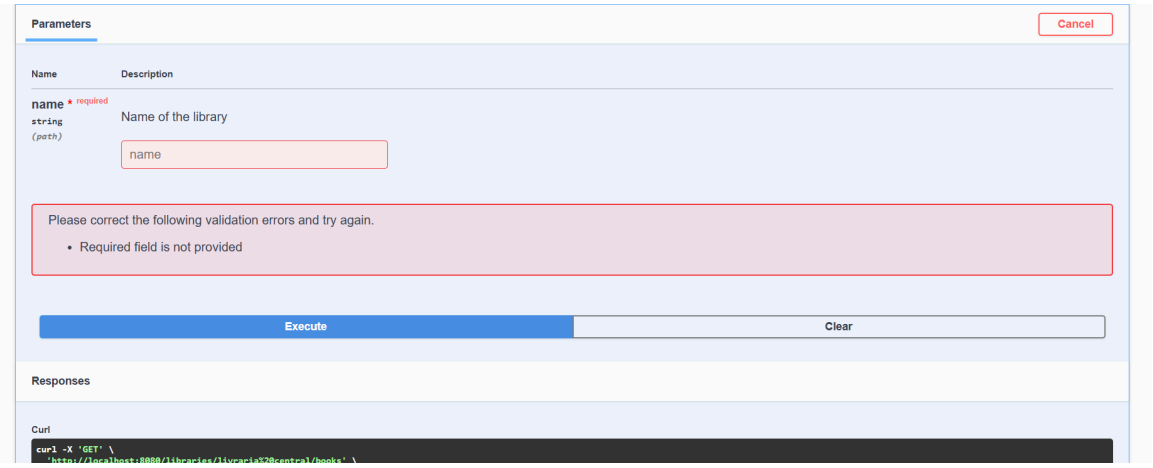






- **Consultar Quantidade de Livros - Ausência de Dados:** O sistema de validação local do Front-end (`required`) age corretamente, impedindo submissões com a caixa de texto vazia.





3. Fluxos Operacionais: PERFIL USUÁRIO (USER)

Nota Técnica de Execução (Espelhamento de Perfis)

Devido a uma falha crítica de segurança na segregação de acessos (Broken Access Control), o sistema atualmente não diferencia as permissões do Usuário Comum em relação ao Administrador. Por conta desta vulnerabilidade, o detalhamento passo a passo dos testes do perfil Usuário Comum não será replicado nesta seção. Constatou-se que **o comportamento da plataforma, as permissões indevidas habilitadas e absolutamente todos os bugs e falhas de integração encontrados espelham integralmente os resultados já documentados no tópico "3.2. Fluxos Operacionais: PERFIL ADMINISTRADOR (Admin)".**

Comportamento Esperado (Regras de Negócio e RBAC): O correto, do ponto de vista arquitetural e de segurança, seria o perfil de Usuário Comum (`user1`) possuir um nível de acesso restrito (Read-Only) em relação às mutações de catálogo. O esperado para este perfil seria o acesso exclusivo para:

- Listar o acervo de livros e as bibliotecas cadastradas;
- Consultar o volume de livros por unidade;
- Visualizar o status de locação (disponibilidade) dos livros;

- Realizar ações limitadas ao próprio escopo, como a atribuição de locatário (reservar/alugar).

Ações estruturais e administrativas — tais como **Criar Livros, Deletar Livros, Criar Bibliotecas e Registrar Pessoas** — deveriam ser completamente ocultadas da Interface de Usuário (Front-end) e, caso forçadas via requisição direta, deveriam ser bloqueadas pelo Back-end (API) com o retorno do status `403 Forbidden` ou `401 Unauthorized`.

[https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-011.pdf e https://github.com/vitoriaMdeA/goledger-challenge-qa/blob/main/qa-reports/BUG-012.pdf]